Generators with `yield` and `next`

A generator function pauses at `yield` and resumes on the next request, producing values lazily.

Your first generator

```
def countdown(n):
    while n > 0:
        yield n          # pause here, hand out n
        n -= 1

gen = countdown(3)       # nothing runs yet!
print(next(gen))
print(next(gen))
print(next(gen))
```

OUTPUT

```
3
2
1
```

```
try:
    next(gen)            # resumes, loop ends
except StopIteration:
    print("done")
```

OUTPUT

```
done
```

for loops call next() for you

```
print(list(countdown(4)))
for x in countdown(2):
    print("x =", x)
```

OUTPUT

```
[4, 3, 2, 1]
x = 2
x = 1
```

Watch the execution flow

```
def trace():
    print("start")
    yield 1
    print("middle")
    yield 2
    print("end")

g = trace()
print(next(g))
print(next(g))
# default instead of error
print(next(g, "exhausted"))
```

OUTPUT

```
start
1
middle
2
end
exhausted
```

Iterable vs iterator vs generator

Term	Meaning
iterable	can be looped over (has `__iter__`): list, str, dict…
iterator	has `__next__`, remembers its position; `iter(x)` makes one
generator	an iterator written with `yield` (or a generator expression)

return inside a generator

```
def two():
    yield 1
    yield 2
    # becomes StopIteration.value
    return "finished"

g = two()
next(g); next(g)
try:
    next(g)
except StopIteration as e:
    print(e.value)
```

OUTPUT

```
finished
```

A practical one: chunking

```
def chunks(seq, size):
    for i in range(0, len(seq), size):
        yield seq[i:i + size]

print(list(chunks("abcdefg", 3)))
```

OUTPUT

```
['abc', 'def', 'g']
```

PITFALL Calling a generator function runs **none** of its body – it only creates the generator. A generator is also **one-shot**: once exhausted, `list(g)` is `[]`; call the function again for a fresh one.

TIP Why bother? Lazy evaluation, tiny memory use, endless sequences, and local variables are remembered between yields for free.